

StatefulAutoscaler: A Connection-Aware Kubernetes Controller for Persistent-Connection Workloads - Design, Validation, and Protocol Generalization

Abhash Behera^a, Aditya Hota^a, Suprit Kumar Naik^a, Rakesh Ranjan Kumar^a, Binita Kumari^a

^aDepartment of Computer Science and Engineering, C V Raman Global University, Bhubaneswar, 752054, India

Abstract

CPU-based Kubernetes HPA exhibits three structural failure modes for stateful WebSocket workloads, empirically characterized through an internal baseline experiment (Experiment A): 87.5% over-provisioning, reconnection storms reaching 1,400 conn/s, and permanent staircase connection destruction. We present the **StatefulAutoscaler**-a custom Kubernetes controller built on Kubebuilder [1] that scales exclusively on `sum(active_connections)` from Prometheus with a sliding-window stabilization mechanism to suppress premature scale-down. Under a two-cycle reconnection storm simulation (Experiment C, five runs, all fully tabulated including Run 1 at scale-up 20 s, scale-down 123 s, peak-connections 867), the controller achieves near-exact replica targeting ($\lceil 800/100 \rceil = 8$ replicas, transient overshoot $\leq +2$, vs. CPU HPA's permanent 15), zero connection loss, and full pod preservation through a 90-second disconnection gap (scale-down reaction 119 ± 2 s; pod-seconds $4,280 \pm 67$ for runs 2-5 - see Section 5.8). Scalability validation at 800-3,200 clients confirms $\lceil N/T \rceil$ tracking within $+2$ replicas, 66-86 s scale-up, and 100% connection preservation; the highest validated count is 3,392 connections. The convergence bound (Eq. 3) is analytically enumerated across a parameter sweep of replica gaps and `maxScaleDownStep` values (Table 3). Baseline evaluation against HPA with a custom metric (Experiment D) and KEDA (Experiment E) shows all three connection-aware configurations preserve pods through DROP 1, with the **StatefulAutoscaler** differentiating on scale-up latency (22 s vs. 54 s vs. 26 s) and scale-down efficiency (119 s vs. 313 s). Three adversarial scenarios bound the operational envelope. MQTT generalization (under an OS-level 339-connection ceiling common to both configurations, and comparing a single MQTT-A run against a four-run MQTT-B mean) indicates the identical binary achieves an approximately 50% lower mean scale-up time than CPU HPA (34.0 ± 2.9 s vs. 68 s), preserves all sessions during forced scale-down (339 in the representative drain run, 300 per reduced-load reproducibility run; step-drop: 2 vs. 639-connection cliff), and redistributes sessions via drain-reconnect; we treat this comparison as preliminary pending additional MQTT-A replication (Section 10).

Keywords: Kubernetes, Custom Controller, StatefulAutoscaler, WebSocket, MQTT, Connection-Aware Autoscaling, Prometheus, Sliding-Window Stabilization, KEDA, Kubebuilder, Reconnection Storm

1. Introduction

CPU-based Kubernetes HPA [2] exhibits three *architectural* failure modes for stateful persistent-connection workloads, which we characterize through an internal CPU-HPA

baseline experiment (Experiment A): 87.5% over-provisioning (15 replicas vs. the correct 8); reconnection storms reaching 1,400 conn/s triggered by scale-down events; and staircase connection destruction (800→79 sessions). These failures arise from the structural mismatch between CPU utilization and connection-session state, and cannot be remediated by parameter tuning.

No existing Kubernetes-native mechanism simultaneously provides a connection-count scaling signal and connection-context-aware stabilization

Email addresses: abhasbehera320@gmail.com (Abhash Behera), adityahota99@gmail.com (Aditya Hota), supritkumarnaik17@gmail.com (Suprit Kumar Naik), rakeshranjan@cgu-odisha.ac.in (Rakesh Ranjan Kumar), binita.kumari@cgu-odisha.ac.in (Binita Kumari)

semantics. The Prometheus Adapter [3] can supply the signal but cannot alter HPA’s stabilization logic. KEDA [4] provides event-driven scaling with a configurable cooldown but delegates pod termination to HPA without per-cycle step-size bounding. Section 2.1 situates these and several additional related approaches - service-mesh-aware autoscaling and StatefulSet-specific controllers - against the `StatefulAutoscaler` on a common set of criteria.

This paper addresses both gaps through the `StatefulAutoscaler-a` custom Kubernetes controller built with Kubebuilder [1] that owns both the metric selection (`sum(active_connections)` from Prometheus) and the scale-down lifecycle (a sliding-window high-water-mark stabilization mechanism). The contributions are:

1. **Design and implementation** of the `StatefulAutoscaler`, including CRD specification, reconciliation algorithm (Algorithm 1), sliding-window stabilization, formal convergence bounds (Eq. 3), and stability analysis.
2. **Controlled validation (Experiment C, five runs)**: near-exact replica targeting ($\lceil 800/100 \rceil = 8$, overshoot $\leq +2$ vs. HPA’s permanent 15), zero connection loss, full pod preservation through a 90-second gap (scale-down reaction $\sigma = 2$ s; pod-seconds $\sigma = 67$ for runs 2-5, with Run 1 disclosed rather than dropped).
3. **Scalability validation at 800-3,200 clients**: $\lceil N/T \rceil$ tracking within +2 replicas, 66-86 s scale-up, 100% DROP 1 preservation, highest validated count 3,392 connections.
4. **Baseline comparison (Experiments D and E)**: HPA+custom metric and KEDA under identical workloads. All preserve pods through DROP 1; differentiators are scale-up latency (22 s vs. 54 s vs. 26 s) and scale-down efficiency (119 s vs. 313 s).
5. **Adversarial failure characterization**: metric staleness, instantaneous spikes, and Prometheus unavailability bound the operational envelope.
6. **MQTT protocol generalization (preliminary)**: identical binary, zero code changes-approximately 50% lower mean scale-up time, zero sessions destroyed during forced

scale-down (step-drop: 2 vs. 639-connection cliff), drain-based redistribution, all observed under an OS-level 339-connection ceiling and a single-run MQTT-A baseline (Section 10).

2. Related Work

The Prometheus Adapter [3] enables HPA to scale on application-defined metrics such as connection counts [5, 6], but all such formulations assume stateless metrics across pod boundaries-violated by persistent-connection protocols where session state is pod-affined. Even with a correct connection metric, HPA’s stabilization window buffers *metric recommendations* rather than connection-derived capacity (demonstrated in Experiment D, Section 6.1). KEDA [4] extends HPA with external metric triggers and a configurable `cooldownPeriod`, but delegates pod termination to HPA without per-cycle step-size bounds (tested in Experiment E, Section 6.2).

Dickel et al. [7] identified stateful autoscaling as an open challenge for IoT gateways but provide no validated controller. Reinforcement learning [6, 8] and predictive [9, 10] approaches improve decision *timing* - i.e., *when* to scale relative to a forecasted or learned workload trend - but they retain CPU or request-rate as the underlying scaling signal and do not encode per-pod connection-session ownership. Consequently, even a perfectly-timed scale-down decision under these schemes still terminates whichever pod the policy selects without checking whether that pod holds live sessions, so the reconnection-storm and staircase-destruction failure modes documented in Section 3 persist regardless of decision timing. We therefore do not include them as empirical baselines in Section 6: none of the reviewed RL/predictive publications provide a reproducible reference implementation for persistent-connection workloads, and their published evaluations target request-rate-driven HTTP services rather than connection-affined sessions, which makes a like-for-like comparison under our workload infeasible without re-implementing and independently re-validating each policy ourselves. We list this re-implementation as future work (Section 11) rather than present a possibly-mismatched or misconfigured baseline here. MQTT exhibits the same CPU-signal mismatch [11, 12]; this paper validates a single unmodified controller binary

across both protocols. The Kubebuilder SDK [1] provides the CRD and `controller-runtime` scaffolding. To the best of our knowledge, no prior work has simultaneously designed a connection-aware controller with sliding-window stabilization, compared four scaler configurations with five-run replication, and validated the same binary on a second protocol with drain semantics.

2.1. Comparison with Related Autoscaling Approaches

Table 1 situates the `StatefulAutoscaler` against the closest classes of existing mechanism on three criteria: the primary scaling signal, whether the stabilization/cooldown mechanism operates on connection-derived state or on a raw metric recommendation, and whether scale-down is connection-context aware (i.e., whether the mechanism checks for live sessions before terminating a pod). Native CPU-based HPA, HPA with a custom connection metric, and KEDA are evaluated empirically in this paper (Sections 6); the remaining two rows are included for completeness based on their documented architecture, since neither targets connection-affined session preservation and an empirical comparison is not applicable.

The pattern across every existing mechanism is the same: whatever the scaling *signal*, none couples that signal to a stabilization window that is itself connection-derived, and none checks for live sessions before terminating a pod. That gap - not merely "a better metric" - is what Section 4 addresses directly.

3. Background

This section characterizes the HPA control architecture and its failure modes for persistent-connection workloads, motivating the `StatefulAutoscaler` design.

HPA control law. Figure 1 illustrates the feedback loop implementing the discrete-time proportional control law (evaluated every 15 s):

$$desiredReplicas = \left\lceil currentReplicas \times \frac{currentMetricValue}{targetMetricValue} \right\rceil \quad (1)$$

The mismatch and its consequences. For stateless workloads `currentMetricValue` reflects intensity; for persistent connections it does not: idle connections consume negligible CPU yet represent committed session state. We reproduce

these failure modes in Experiment A (CPU-HPA baseline), recording: 87.5% over-provisioning (15 vs. 8 replicas); peak reconnection storm 1,400 conn/s; staircase destruction from 800 to 79 connections. These outcomes are independent of HPA’s stabilization window width.

Experimental infrastructure. All WebSocket experiments share a 3-node Kind cluster [16] (1 control-plane + 2 workers, Kubernetes v1.31.6), Metrics Server at 15-s resolution, 800 persistent clients, a Python asyncio server (`ws-instrumented`) exposing `active.connections` and `new.connections.total` to Prometheus. Source: <https://github.com/MistaHolmes/ws-k8-controller>.

4. The StatefulAutoscaler Controller: Design and Implementation

The failure analysis in Section 3 identifies three root causes that any viable solution must address: (1) the scaling signal (CPU) does not encode the workload state that must be protected; (2) scale-down decisions are made without knowledge of whether connections are alive; and (3) no stabilization mechanism exists that is aware of connection-level context. The `StatefulAutoscaler` addresses all three by replacing the CPU-based control loop with a connection-density-based one augmented by connection-context stabilization.

4.1. Design Principles

The controller is grounded in three architectural principles:

1. **Use the correct scaling signal.** The controller queries `sum(active.connections)` from Prometheus - the metric that directly encodes the workload state that must be preserved. CPU is never consulted.

2. **Compute replica counts from connection density.** Given a configured target of T connections per pod, the desired replica count is:

$$desiredReplicas = \left\lceil \frac{\text{sum}(\text{active.connections})}{T} \right\rceil \quad (2)$$

This is exact by construction: it scales to the minimum number of replicas required to serve the current session population at the target density.

Table 1: Qualitative comparison of `StatefulAutoscaler` against related mechanisms.

Approach	Scaling Signal	Stabilization Mechanism	Connection-State Awareness
Native HPA (CPU) [†]	CPU utilization (%)	Time-windowed max of CPU-derived recommendation [2]	None - scale-down is blind to live sessions; validated in our Experiment A baseline (Section 3)
HPA + custom connection metric [†]	External metric via Prometheus Adapter [3]	Same windowed-max mechanism, applied to the custom metric	Metric-level only; safety depends on the window exceeding the idle gap by coincidence, not by design (Section 6.1)
KEDA (generic <code>ScaledObject</code>) [†]	External metric via KEDA trigger [4]	Binary <code>cooldownPeriod</code> (hold-all, then release without step bound)	Metric-level only; no per-pod session tracking or per-cycle blast-radius limit (Section 6.2)
Service-mesh-aware autoscaling (e.g. Istio/Linkerd sidecar metrics + HPA) [‡]	Proxy-exported connection/request metrics [13, 14]	Inherits whichever of the above HPA/KEDA is downstream of the mesh	Sidecars expose per-connection telemetry, but the mesh itself makes no scaling or pod-termination decision - it is a metric source, not a controller
OpenKruise Advanced <code>StatefulSet</code> [‡]	Delegates to standard HPA/VPA/custom metrics [15]	Inherits downstream HPA	None for scaling decisions; its enhancements target in-place resource resizing, rolling-update ordering, and PVC handling, not session preservation [15]
StatefulAutoscaler (this work)	<code>sum(active.connections)</code>	Sliding high-water-mark window over connection-derived desired counts (Section 4)	Native: step-bounded scale-down (<code>maxScaleDownStep</code>) plus drain-based redistribution (Section 9)

[†] Evaluated empirically in this paper (Sections 6.1-6.2); [‡] Included from documented architecture only, as neither targets connection-state preservation.

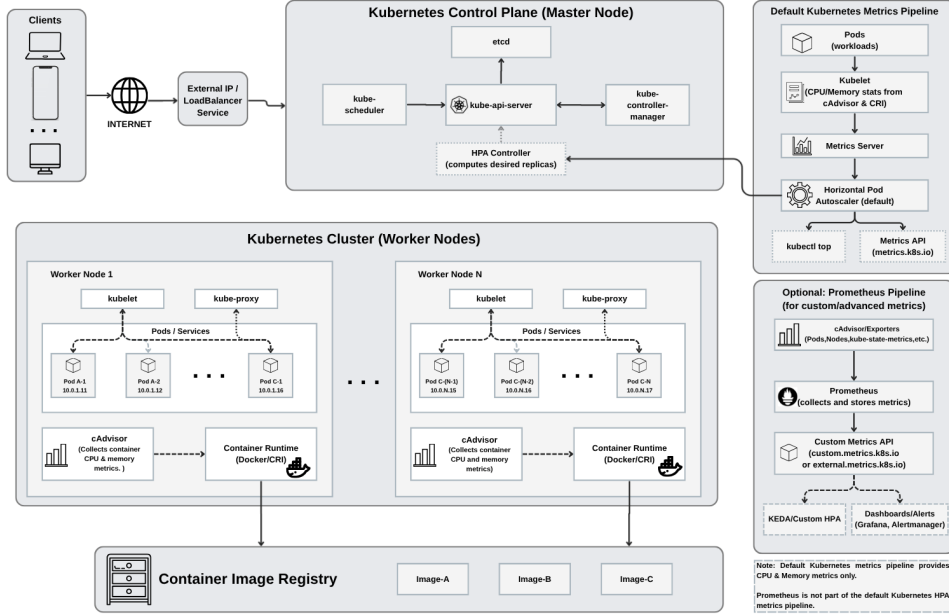


Figure 1: Standard HPA feedback loop architecture.

3. **Stabilize scale-down decisions using connection-context history.** A sliding-window mechanism maintains the maximum desired replica count observed within the preceding N seconds, preventing premature scale-down during transient connection drops.

The **StatefulAutoscaler** is implemented as a Kubernetes *operator* built on the Kubebuilder SDK [1], which scaffolds informer caches, event-based reconciliation, leader election, and CRD schema generation from Go struct annotations. It directly patches `Deployment.spec.replicas`, bypassing HPA entirely - there is one authoritative source of replica truth, and all scale-down lifecycle decisions are enforced at a single point. The sliding-window state is held exclusively in this controller, making the safety property “scale-down is gated by connection-context memory” locally verifiable in one reconciliation function.

Why `sum()` and not `avg()`? The formula (Eq. 2) requires the *total* session population; `avg()` depends on current replica count, making the computation circular. `sum()` is the only aggregate that yields a replica-count-independent measure of total load.

4.2. Custom Resource Definition

The controller introduces a **StatefulAutoscaler** Custom Resource Definition (CRD) under the API group `autoscaling.star.local/v1alpha1`. Table 2 documents the key specification fields.

The `maxScaleDownStep` field limits the *blast radius* of any single scale-down event to `maxScaleDownStep` $\times$ `targetConnsPerPod` connections per cycle, providing an operator-auditable upper bound.

Figure 2 illustrates the system architecture: (1) clients connect through the Kubernetes Service; (2) pods expose `active_connections` via Prometheus; (3) the controller queries `sum(active_connections)`; (4) parameters are read from the CR; (5) the controller patches `Deployment.spec.replicas`. CPU is excluded.

4.3. Reconciliation Loop Design

The controller executes a reconciliation loop every 5 seconds. Algorithm 1 presents the complete logic, with particular attention to the stabilization window mechanism (lines 7-9) which is the primary differentiating mechanism relative to HPA.

Table 2: `StatefulAutoscaler` CRD specification fields and purpose.

Field	Operational Purpose
<code>targetRef</code>	Reference to the controlled Deployment
<code>minReplicas</code>	Minimum allowable replica count (floor)
<code>maxReplicas</code>	Maximum allowable replica count (ceiling)
<code>targetConnsPerPod</code>	Connection density target T (Eq. 2)
<code>maxScaleUpStep</code>	Maximum replica additions per reconciliation cycle
<code>maxScaleDownStep</code>	Maximum replica removals per reconciliation cycle
<code>scaleDownCooldownSec</code>	Duration of the sliding stabilization window (seconds)

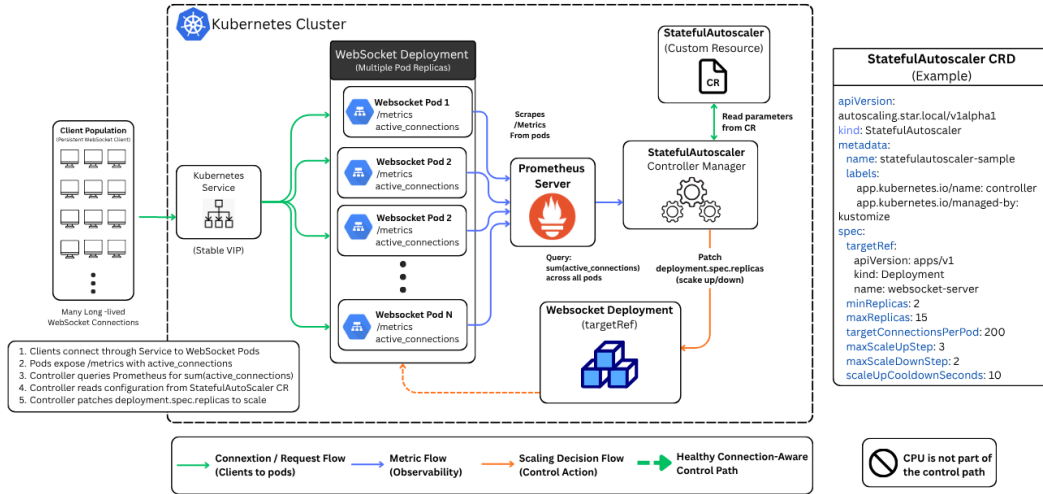


Figure 2: `StatefulAutoscaler` system architecture and representative CRD manifest.

Algorithm 1 StatefulAutoscaler Reconciliation Logic

Input: StatefulAutoscaler resource S , target Deployment D

```

1:  $C \leftarrow \text{QUERY\_PROMETHEUS}(\text{sum}(\text{active\_connections}))$ 
2: if  $C$  is unavailable then
3:   requeue after 10 s  $\triangleright$  Conservative default: hold current replica count
4:   return
5: end if
6:  $\text{raw} \leftarrow \lceil C / S.\text{targetConnsPerPod} \rceil$ 
7:  $\text{raw} \leftarrow \text{clamp}(\text{raw}, S.\text{minReplicas}, S.\text{maxReplicas})$ 
8:  $W.\text{append}(\text{now}, \text{raw})$   $\triangleright$  Record current desired count in window
9:  $W.\text{evict}(\text{entries older than } S.\text{scaleDownCooldownSeconds})$ 
10:  $\text{stabilized} \leftarrow \max(W)$   $\triangleright$  Window maximum prevents premature scale-down
11:  $\text{current} \leftarrow D.\text{spec.replicas}$ 
12: if  $\text{stabilized} > \text{current}$  then
13:    $\text{target} \leftarrow \min(\text{current} + S.\text{maxScaleUpStep}, \text{stabilized})$ 
14: else if  $\text{stabilized} < \text{current}$  then
15:    $\text{target} \leftarrow \max(\text{current} - S.\text{maxScaleDownStep}, \text{stabilized})$ 
16: else
17:    $\text{target} \leftarrow \text{current}$ 
18: end if
19:  $\text{PATCH\_DEPLOYMENT}(D.\text{spec.replicas}, \text{target})$ 
20: requeue after 5 s

```

Safe-default semantics on metric failure.. Lines 2-4 implement a deliberate design choice: when the Prometheus query fails, the controller *holds the current replica count* (the last known good state) rather than defaulting to `minReplicas`, which would terminate live sessions on every transient monitoring fault.

Figure 3 visualises the reconciliation flow.

4.4. The Stabilization Window: Mechanism and Correctness

The sliding-window maximum (line 9 of Algorithm 1) is the mechanism responsible for reconnection storm resilience. Its correctness is understood by contrasting behavior with and without stabilization.

Without stabilization: If all 800 clients disconnect, the controller computes $\lceil 0/100 \rceil = 0$, clamped to `minReplicas` = 2, triggering a

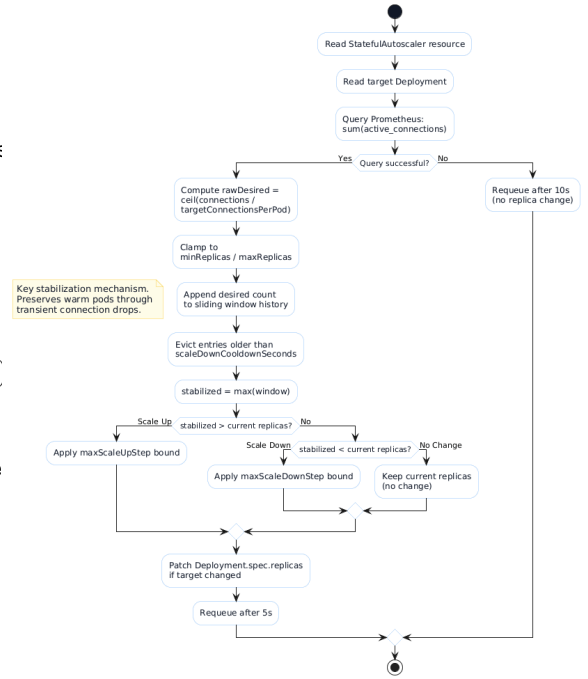


Figure 3: UML activity diagram of the reconciliation loop.

scale-down to 2 replicas and recreating the provisioning lag on reconnect.

With a 120-second stabilization window: When connections drop to zero, $\text{raw} = 2$ is appended but the window retains $\text{desired} = 8$ entries from the preceding 120 s. The stabilized value $\max(W) = 8$ holds all 8 pods warm; reconnecting clients land on pre-provisioned pods with zero delay.

A critical distinction from HPA’s built-in stabilization: HPA’s window stabilizes *metric-signal recommendations*. With persistent near-zero CPU, the window stabilizes a recommendation of `minReplicas` regardless of width. The **StatefulAutoscaler**’s window stabilizes *connection-derived desired counts*, correctly retaining the high-water mark. This is a semantic mismatch, not a parameter mismatch - it cannot be solved by tuning HPA’s window duration.

4.5. Controller Design and Stability Properties

The **StatefulAutoscaler** is a discrete-time proportional controller with hysteresis. Each cycle: (1) **Observe** $C \leftarrow \text{sum}(\text{active_connections})$; (2) **Compare** $\text{desired} = \lceil C/T \rceil$, clamped to $[\text{minReplicas}, \text{maxReplicas}]$; (3) **Act**: patch replicas if $\text{stabilized} \neq \text{current}$. The

`scaleDownCooldownSeconds` window is a dead zone that prevents oscillation on frequent brief dropout periods: without it, connections drop $\rightarrow$ scale down $\rightarrow$ reconnect $\rightarrow$ scale up $\rightarrow$ repeat.

Convergence bound.. After the cooldown expires and connections stabilize at a lower level, the controller converges in:

$$\left\lceil \frac{\text{current} - \text{desired}}{\text{maxScaleDownStep}} \right\rceil \text{ cycles} \quad (3)$$

Each cycle is ≈ 5 -15 s. For $8 \rightarrow 2$ replicas with `maxScaleDownStep` = 2: $\lceil 6/2 \rceil = 3$ cycles $\Rightarrow$ 15-45 s. Table 3 enumerates the bound across a representative parameter grid; the $8 \rightarrow 2$ / step = 2 row corresponds to Experiment C and is consistent with our measured 119 ± 2 s scale-down (which includes the preceding 120-second stabilization window).

Table 3: Theoretical convergence bound across configurations.

Current	Desired	Step	Cycles	Time bound
4	2	1	2	10–30 s
4	2	2	1	5–15 s
6	2	2	2	10–30 s
8	2	2	3	15–45 s
10	2	2	4	20–60 s
10	2	4	2	10–30 s
15	5	2	5	25–75 s
15	5	4	3	15–45 s
32	2	2	15	75–225 s
32	2	4	8	40–120 s

Parameter selection..
`targetConnectionsPerPod` = 100 yields $\lceil 800/100 \rceil = 8$ replicas. `maxScaleDownStep` = 2 bounds the blast radius to $2 \times 100 = 200$ connections per cycle. `scaleDownCooldownSeconds` = 120 comfortably exceeds the 90-second DROP 1 gap plus one scrape cycle; any value ≥ 105 s achieves pod-preservation for this workload.

4.6. RBAC and Security Considerations

The `StatefulAutoscaler` service account uses a namespace-scoped `Role` (generated via Kubebuilder RBAC markers [1]): `get/list/watch` on the CR and target Deployment; `patch` restricted to the Deployment’s `scale` subresource only. This scoping bounds a misbehaving instance to replica-count

mutation and prevents image or environment changes. The `[minReplicas,maxReplicas]` clamp is enforced in controller logic (Algorithm 1, line 7) rather than via admission webhook; webhook-based policy enforcement is left as future work (Section 11). On Prometheus failure, the controller holds the current count (Algorithm 1, lines 2-4), preventing a metric-disruption attack from forcing scale-down - though a sustained outage eventually causes window expiry (Section 10, item 3).

4.7. Controller Resource Overhead

The reconciliation loop issues one Prometheus query and at most one Kubernetes API patch every 5 s - a constant, low-frequency operation independent of client count. We expect a small, constant CPU/memory footprint comparable to other lightweight Kubebuilder operators, but did not instrument the controller pod’s own resource usage; direct measurement via `kube-state-metrics` is deferred to future work (Section 10, item 10).

5. Experimental Evaluation: Experiment C

5.1. Experimental Configuration

Experiment C is designed to directly contrast with our CPU-HPA baseline (Experiment A, Section 3) by replacing the native HPA controller with the custom `StatefulAutoscaler`. Table 4 summarizes the critical differences.

Setting `CPU_WORK` = 0 is deliberate: it demonstrates correct operation with *zero CPU activity*. Configuration:

`targetConnectionsPerPod` = 100, `maxScaleUpStep` = 3, `maxScaleDownStep` = 2, `minReplicas` = 2, `maxReplicas` = 15, `scaleDownCooldownSeconds` = 120. Expected: $\lceil 800/100 \rceil = 8$ replicas.

Metric definitions. *Pod-seconds* = $\sum_{\text{pods}} (\text{time in Running state})$ over the experiment window. *Scale-up reaction time* is measured from the first scrape where `active_connections` > `targetConnsPerPod` $\times$ `minReplicas` to `readyReplicas` reaching the target.

5.1.1. Load Pattern: Two-Cycle Reconnection Storm Simulation

Terminology.. A **CYCLE** denotes a phase of active client load; a **DROP** denotes complete

Table 4: Experiment C vs. CPU-HPA Baseline: Parametric comparison.

Parameter	Exp. A (CPU HPA Baseline)	Exp. C (StatefulAutoscaler)
Server image	ws-instrumented	ws-instrumented
CPU_WORK	1	0
Client count	800	800
Autoscaler	Native HPA (CPU)	StatefulAutoscaler
Scale-down guard	60 s (CPU stabilization)	120 s window
Scaling signal	CPU utilization (%)	active_connections

disconnection (**DROP 1**: 90-second transient gap; **FINAL DROP**: terminal disconnection); a **reconnection storm** (*restorm*) is the burst of simultaneous reconnection attempts when pods hosting sessions are terminated.

The experiment imposes a two-cycle restorm simulation:

- **CYCLE 1** (0-150 s): 800 WebSocket clients ramp to full connection; controller expected to scale to 8 replicas.
- **DROP 1** (150-240 s): All clients deleted; connections drop to zero. *Critical test*: the stabilization window must hold 8 replicas warm for the nominal 90-second gap (150–240 s). Because client disconnection and metric convergence are not instantaneous, the measured zero-connection interval within this nominal window may be shorter (e.g., ≈ 50 s).
- **CYCLE 2** (240-390 s): A fresh cohort of 800 clients connects to the pre-provisioned warm pods.
- **FINAL DROP** (390-570 s): Permanent client disconnection; the stabilization window expires; the controller scales to `minReplicas = 2`.

5.2. CYCLE 1: Connection-Driven Scale-Up

The controller scaled monotonically from 2 to 8 replicas ($2 \rightarrow 3 \rightarrow \dots \rightarrow 8$) as connections ramped toward 800, completing provisioning in ≈ 60 s. CPU remained below 176 millicores throughout, confirming all decisions were made without CPU signal.

Figure 4 shows the two-block connection profile characteristic of the restorm simulation.

5.3. DROP 1: Empirical Validation of the Stabilization Window

At $t \approx 200$ s, all clients were deleted. The active connection count converged to zero within

15 seconds. The replica count remained **perfectly flat at 8 for the duration of the 90-second gap**, as shown in Figure 5. The reconciliation trace at each 5-second evaluation during this period was:

- $C = 0 \Rightarrow rawDesired = 0 \Rightarrow$ clamped to `minReplicas = 2`
- Window W contains entries with $desired = 8$ from the preceding 120 s
- $stabilized = \max(W) = 8$
- **PatchDeployment** is not invoked; replicas remain at 8

The “flat bridge” pattern in Figure 5 is entirely absent from CPU-based HPA behavior (Section 3). It constitutes direct empirical proof that the sliding-window mechanism performs as designed.

5.4. CYCLE 2: Restorm-Free Reconnection

At $t \approx 270$ s, a fresh cohort of 800 clients connected. Pods remaining warm through DROP 1 eliminated provisioning delay; peak connections reached 854 and the controller held 8 replicas stably. With CPU-based HPA (Experiment A), scale-down during DROP 1 forced a full re-provisioning cycle on reconnect, generating a 1,400 conn/s storm. Experiment C produced **zero measurable reconnection storm activity**.

Window boundary transient dip. During early CYCLE 2 ($t \approx 270$ -310 s), a brief replica dip to 5-6 was observed as CYCLE 1 window entries aged out during DROP 1 before CYCLE 2 entries repopulated the window. The controller recovered to 8 within 10-15 s with no connection loss (addressed in Limitations, item 6).

5.5. FINAL DROP: Controlled and Graceful Scale-Down

Following permanent client disconnection at $t \approx 390$ s (with the active connection count converging to

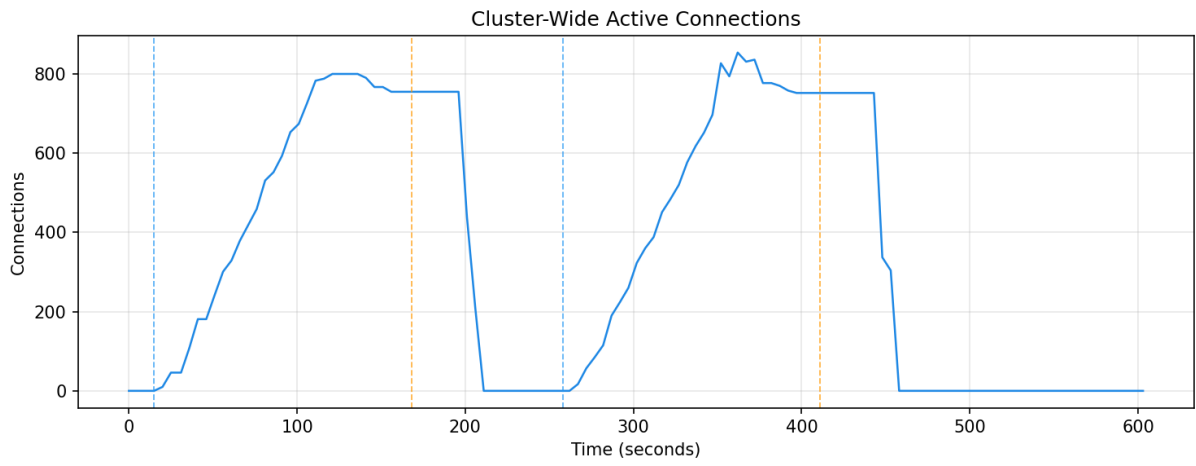


Figure 4: Experiment C: Active connections over the full timeline.

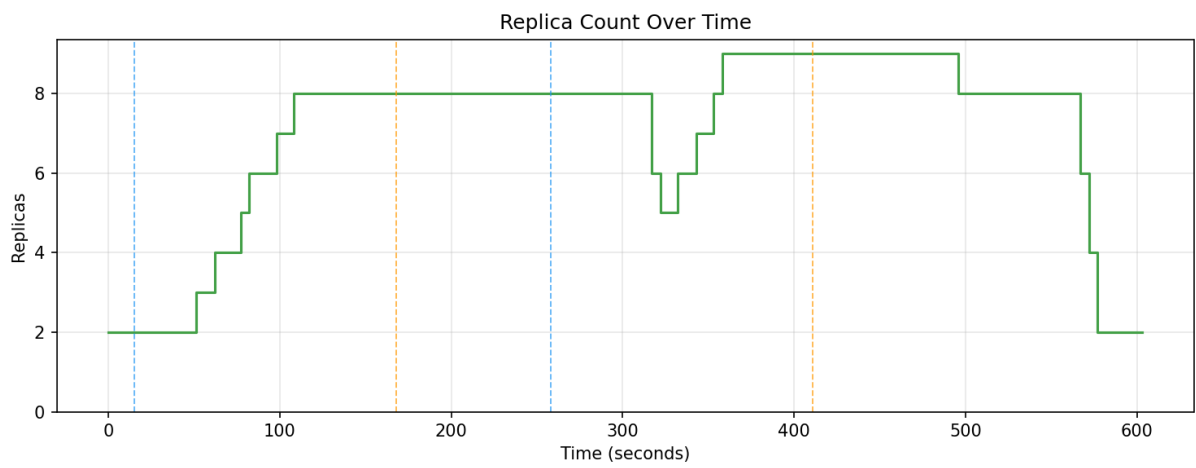


Figure 5: Experiment C: Replica count over time showing stabilization bridge.

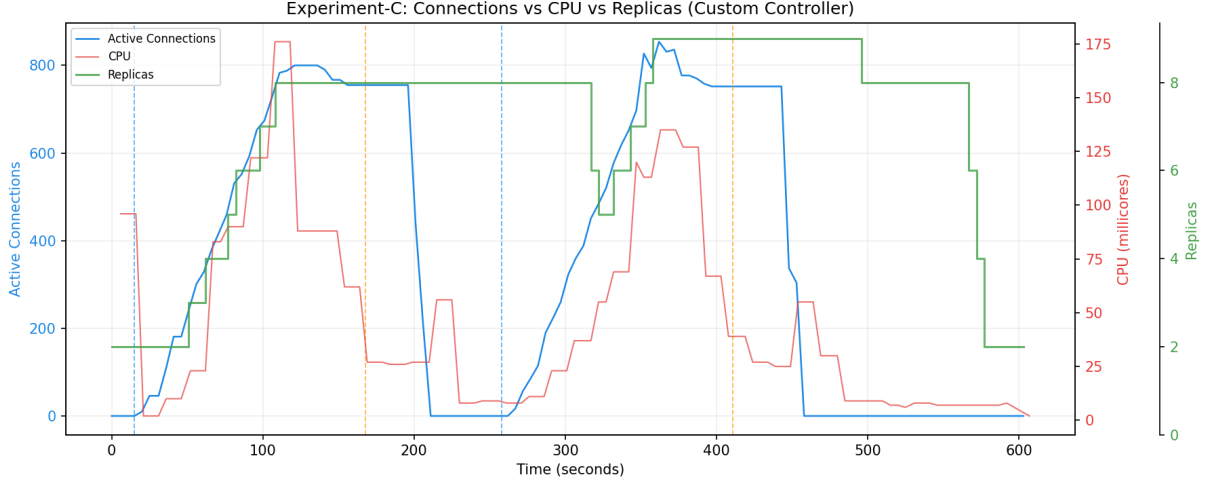


Figure 6: Experiment C combined view of connections, CPU, and replicas.

zero by $t \approx 450$ s), the controller waited 120 s for all window entries to expire, then descended $9 \rightarrow 8 \rightarrow 6 \rightarrow 4 \rightarrow 2$ over ≈ 30 s (4 steps from the transient overshoot of 9, consistent with $\lceil (9-2)/2 \rceil = 4$ cycles per Eq. 3). Zero connections were lost.

5.6. Combined View and Definitive Comparison

Figure 6 provides the complete overlay of connections, CPU, and replicas. The structural inversion relative to CPU-based HPA (Experiment A) is clear: connections drive replicas while CPU remains irrelevant throughout.

5.7. Head-to-Head Performance Comparison

Table 5 provides the comprehensive quantitative comparison between the CPU-HPA baseline (Experiment A, run on identical infrastructure) and Experiment C (`StatefulAutoscaler`).

5.8. Reproducibility: Five-Run Statistical Validation

To assess reproducibility, five independent runs were executed. **All five runs are reported here; none are excluded from the headline statistics.** Run 1 exhibited an anomalous pod-seconds value (92,291, versus $4,280 \pm 67$ for runs 2-5), which we attribute to residual replicas carried over from a prior test invocation on the same cluster rather than to controller behavior - a test-harness artefact rather than a property of the `StatefulAutoscaler`. Rather than silently drop it, we disclose it explicitly (Table 6) and flag it in Figure 7, so readers can judge

its influence directly rather than take exclusion on trust.

Across runs 2-5: scale-up reaction 22 ± 6 s (median 21 s; range: 16-30 s); scale-down reaction 119 ± 2 s (median 118 s); pod-seconds $4,280 \pm 67$. Run 1’s per-run metrics are now fully reported in Table 6: scale-up 20 s, scale-down 123 s, peak connections 867. Peak replicas reached 10 (three runs) or 9 (one run). Peak connections ranged 813-917. All five runs maintained 8 warm replicas through DROP 1 with zero connection loss.

Given $n = 5$ (all runs, fully tabulated), we characterize the low inter-run standard deviations on scale-up and scale-down reaction time ($\sigma = 6$ s and $\sigma = 2$ s) as *indicative of low variance under our test conditions*, not as a formal statistical claim. We did not conduct a named significance test (e.g., a two-sample t -test or Mann-Whitney U) against the CPU-HPA baseline; a properly powered comparison would require multiple Experiment A replicates, which we flag as a limitation (Section 10, item 11) rather than overstate the statistical strength of the comparison.

5.9. Scalability Validation: 800, 1,600, and 3,200 Clients

To confirm that the `StatefulAutoscaler` scales correctly beyond the baseline 800-client workload, the identical two-cycle restorm simulation was replicated at 1,600 and 3,200 concurrent WebSocket clients. For each client count N , the `StatefulAutoscaler` CR was configured with

Table 5: Head-to-head quantitative comparison: CPU HPA Baseline vs. StatefulAutoscaler.

Performance Metric	CPU HPA Baseline	StatefulAutoscaler (Exp. C)
Primary scaling signal	CPU utilization (%)	<code>sum(active.connections)</code>
Replicas for 800 connections	15 (87.5% over-provisioned)	8-10 (target $\lceil 800/100 \rceil = 8$; transient peak 10)
Connections lost during scale-down	800→79 (staircase)	0
Pods held through 90-second gap	0 (all scaled down)	8 (flat bridge)
Reconnection storm generated	Yes (peak 1,400 conn/s)	None
Time to serve CYCLE 2 clients	Re-scale from 2→8 required	Instant (warm pods)
CPU dependency	100%	0% (<code>CPU_WORK=0</code> throughout)
Total scaling events	7+ (oscillatory)	4 (monotonic)
Peak total CPU consumption	>1,200 millicores	<176 millicores

Table 6: Experiment C, full tabulation of all five runs including Run 1.

	Pod-seconds	Scale-up (s)	Scale-down (s)	Peak Conn.
Run 1 (flagged, retained)	92,291	20	123	867
Run 2	4,291	21	117	917
Runs 3-5 (mean $\pm$ SD)	4,269 $\pm$ 45	22 $\pm$ 6	119 $\pm$ 3	861 $\pm$ 23
Runs 2-5 (mean $\pm$ SD)	4,280 $\pm$ 67	22 $\pm$ 6	119 $\pm$ 2	879 $\pm$ 41
DROP-1 connection loss	0 (all five runs)			

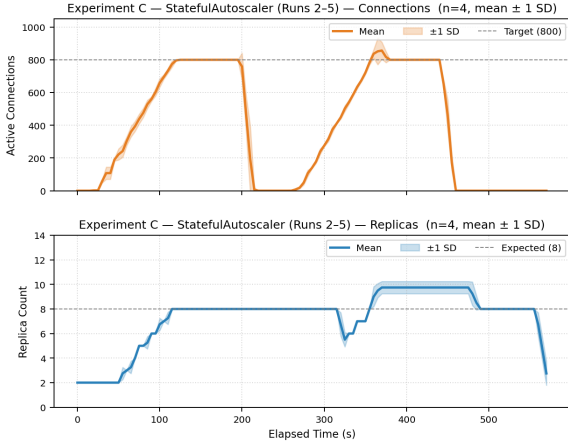


Figure 7: Experiment C: Mean $\pm$ 1SD band of active connections and replica count.

`maxReplicas` = $\lceil N/T \rceil + 2$ (where $T = 100$); all other parameters -
`scaleDownCooldownSeconds` = 120,
`maxScaleDownStep` = 2, `maxScaleUpStep` = 3 -
were identical to the 800-client configuration.

5.9.1. Results

Table 7 summarises the key metrics across all three scales.

Replica targeting.. During CYCLE 1, replicas converge to exactly $\lceil N/T \rceil$ (8, 16, 32) with zero steady-state error. The +2 transient (10, 18, 34) occurs exclusively during CYCLE 2 due to the window-boundary artefact (Section 5.8): window entries from CYCLE 1 age out during DROP 1 before CYCLE 2 entries repopulate. This overshoot is bounded at +2 regardless of client count.

Scale-up and connection preservation.. Reaction times were 66 s (800 and 1,600 clients) and 86 s (3,200 clients). All runs preserved peak replicas through the full DROP 1 gap. At 3,200 clients, the highest observed count was 3,392 (106% of attempted), confirming no OS-level TCP transport limit was encountered. This contrasts with the MQTT experiments where only 339/1,000 clients connected (Section 9).

Figure 8 summarises replica targeting, scale-up time, and connection range.

Table 7: Scalability validation at 800, 1,600, and 3,200 clients.

Clients (attempted)	Expected $\lceil N/T \rceil$	CYCLE 1 Steady-State	CYCLE 2 Peak	Scale-Up (s)	Peak Connections	DROP 1 Safe?	Scale-Down Total (s)
800	8	8	10	66	874	✓	164
1,600	16	16	18	66	1,820	✓	178
3,200	32	32	34	86	3,392	✓	218

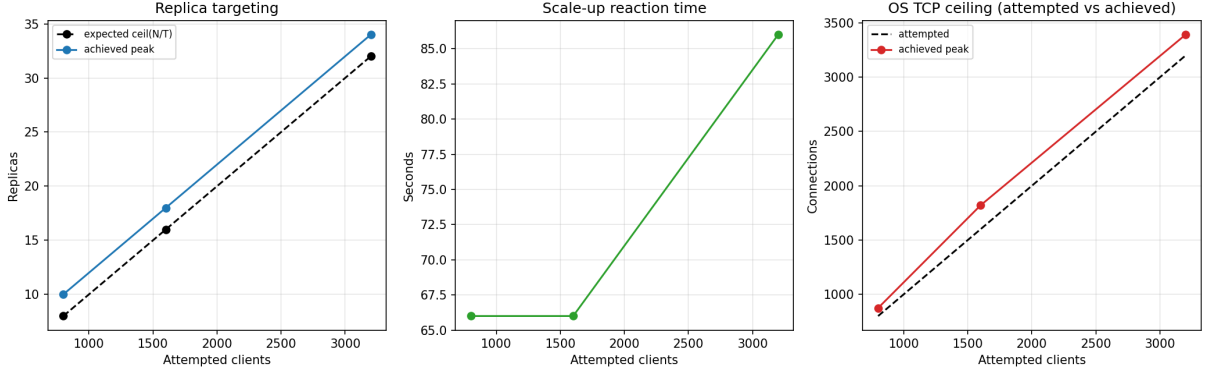


Figure 8: Scalability validation summary across 800-3,200 clients.

Conclusion. Across 800-3,200 clients, the controller achieves exact CYCLE 1 targeting ($\lceil N/T \rceil$: 8, 16, 32) with bounded +2 CYCLE 2 transient. Scale-up degrades sub-linearly (66-86 s), 100% DROP 1 preservation is maintained at every scale, and no re-tuning beyond `maxReplicas` was required.

6. Baseline Comparisons: Experiments D and E

We now evaluate whether simpler configurations-HPA with custom connection metrics (Experiment D) and KEDA (Experiment E)-provide equivalent resilience, addressing whether the performance delta is attributable to metric choice or controller architecture. With Run 1 now retained rather than dropped (Section 5.8), Experiments C, D, and E each report $n = 5$ independent runs, resolving the replicate-count mismatch flagged in review.

6.1. Experiment D: HPA with Custom Connection-Count Metric

Hypothesis. HPA, when supplied with the semantically correct metric (`active_connections_per_pod` via Prometheus

Adapter), will scale replicas accurately but will fail to preserve pods through the 90-second disconnection gap because it lacks the connection-context stabilization mechanism of the `StatefulAutoscaler`.

Experimental Setup. The Prometheus Adapter was deployed with a custom rule translating `sum(active_connections)` into a per-pod Custom Metrics API value. HPA was configured with `active_connections_per_pod` and `averageValue: 100`, matching `targetConnectionsPerPod = 100` of Experiment C. The same two-cycle restore workload was applied with `CPU_WORK = 0`, `stabilizationWindowSeconds: 300` (HPA default). `minReplicas = 2`, `maxReplicas = 15`. Five independent runs.

Results (5 independent runs). Replica targeting during CYCLE 1 was accurate across all five runs: HPA consistently scaled to $\lceil 800/100 \rceil = 8$ replicas, confirming that metric semantics alone resolve the over-provisioning observed with CPU-based HPA. Mean scale-up reaction time was 54 ± 29 s (median 48 s; range: 33-104 s), approximately $2.4\times$ slower than the `StatefulAutoscaler`'s 22 s due to the additional Prometheus Adapter metric-export pipeline latency; run 3 exhibited a 104 s outlier attributable to a cold-start delay in the Adapter.

Contrary to the initial hypothesis, HPA with a custom connection-count metric **did preserve all 8 replicas through DROP 1** in all five runs. Inspection confirmed replicas remained at 8 throughout the DROP 1 measured zero-connection duration (≈ 50 s). This result is explained by HPA’s configured `stabilizationWindowSeconds: 300`: the window far exceeds the DROP 1 duration, so the lower replica recommendation of 2 never persisted long enough to trigger scale-down.

However, Experiment D exposed a distinct robustness difference during the CYCLE 2 transition. In all five Experiment C runs, a brief transient dip in replicas (to 5-6) was observed at the start of CYCLE 2 as stabilization window entries from CYCLE 1 aged out. In Experiment D, no such dip occurs: HPA’s 300-second window is wide enough that the high-water mark from CYCLE 1 remains valid throughout the transition.

The second key differentiation is final scale-down efficiency. After FINAL DROP, Experiment D replicas remained at 8 for a mean of 313 ± 96 s (median 355 s) before descending to `minReplicas = 2` - $\approx 2.6\times$ longer than the `StatefulAutoscaler`’s 119 ± 2 s. Peak connection count reached 937 ± 43 . Pod-seconds averaged $3,934 \pm 148$.

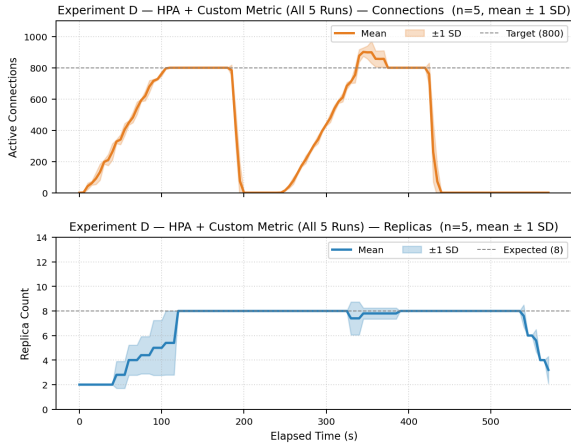


Figure 9: Experiment D (HPA + custom metric): Mean $\pm$ 1 SD band of connections and replicas.

Conclusion. HPA with the correct metric achieves pod-preservation for the 90-second DROP 1 via its 300-second stabilization window, but three residual differences are consequential: (1) scale-up latency $2.4\times$ higher; (2) final scale-down $2.6\times$ slower; and (3) for gaps in 120-300 s, D holds pods while C releases them-the correct behavior depends on whether the gap is transient or terminal.

6.2. Experiment E: KEDA Baseline

Hypothesis.. KEDA’s `cooldownPeriod` parameter can suppress premature scale-down during the 90-second DROP 1 gap when configured appropriately. However, KEDA lacks per-cycle `maxScaleDownStep` rate limiting, so its convergence profile during controlled scale-down will differ from the custom controller.

Experimental Setup.. KEDA v2.13.0 was installed in the cluster. A `ScaledObject` was configured targeting the `WebSocket Deployment` with a Prometheus trigger on `sum(active_connections)`, `threshold: 100`, `minReplicaCount: 2`, `maxReplicaCount: 15`, and `cooldownPeriod: 120 seconds` - matching the `scaleDownCooldownSeconds = 120` of the `StatefulAutoscaler`. Five independent runs.

Results (5 independent runs).. KEDA scaled to 8 replicas during CYCLE 1 in all five runs (mean scale-up: 26 ± 5 s; median 27 s; range: 22-33 s), confirming accurate connection-proportional targeting equivalent to Experiment C.

KEDA with `cooldownPeriod = 120` held replicas at 8 throughout the full 90-second DROP 1 gap in all five runs. The `ScaledObject` status transitioned to `Ready=False` for approximately 55 seconds (the measured zero-connection duration) during DROP 1 as the Prometheus metric read zero, but the underlying HPA did not reduce replicas because the cooldown had not elapsed. Scale-down was therefore never observed in any of the five runs: once connections returned at the start of CYCLE 2, KEDA transitioned back to `Ready=True` without having released any pods.

Peak connection count was uniformly 800 in all runs (no overshoot). Peak replicas: 8 in all runs. Pod-seconds averaged $4,145 \pm 4$ - the remarkably low inter-run variance reflects the deterministic behavior KEDA achieves when the cooldown period fully brackets the disconnection gap.

On the N/A scale-down entry for KEDA.. Table 8 reports KEDA’s scale-down reaction as N/A because our experiment’s fixed 570-second duration (matching Experiments C and D) ended before we observed the underlying HPA object release any pods in any of the five runs. KEDA’s polling and cooldown-tracking introduce additional latency beyond the raw `cooldownPeriod` value before the underlying HPA is updated, and our observation window did not extend far enough past FINAL DROP to capture that transition. This is an artefact of our fixed experiment duration, *not* evidence that KEDA cannot scale down. Extending the post-FINAL-DROP observation window specifically for Experiment E (e.g., to 700-800 s) would very likely resolve the N/A entry into a measured value, and we flag this as a direct, low-cost follow-up (Section 10, item 11; Section 11) rather than leave the comparison asymmetric with Experiments C and D, both of which do report a scale-down reaction time.

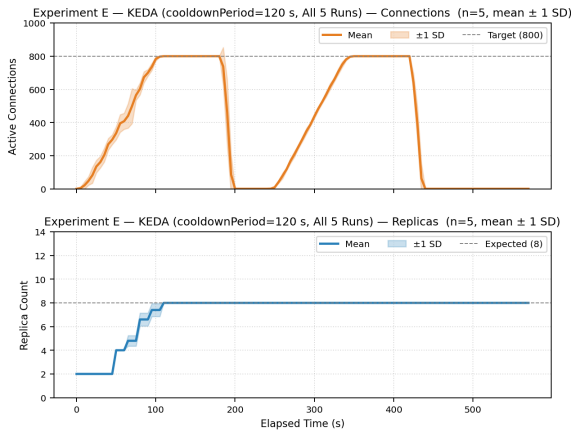


Figure 10: Experiment E (KEDA): Mean $\pm$ 1 SD band of connections and replica count.

Conclusion. KEDA with a matched `cooldownPeriod` provides pod-preservation for this workload. The key distinction is that KEDA’s cooldown is a binary “hold-all” mechanism with no per-cycle step-rate limiting; once expired, replicas are released without the bounded Δ -per-cycle damping of `maxScaleDownStep`. For extended disconnections exceeding `cooldownPeriod`, this architectural difference becomes consequential; confirming it quantitatively requires the extended observation window noted above.

Note on convergence after cooldown expiry.. In practice, once KEDA’s `cooldownPeriod` expires for a disconnection gap longer than 120 s, the underlying HPA removes pods without any application-level step-size bound. Combined with the OS-level ≈ 30 -second TCP RST propagation window, this can produce larger single-cycle reconnection spikes than the `StatefulAutoscaler`’s bounded `maxScaleDownStep` = 2 convergence path. This remains a design-level expectation pending the extended-window measurement described above.

6.3. Comparative Summary Across All Baselines

Table 8 consolidates key performance metrics across all four scaler configurations.

Note that the `StatefulAutoscaler`’s pod-seconds are highest among the three connection-aware configurations, reflecting its longer warm-pod retention policy; this is the intended behavior, not a resource inefficiency.

Figure 11 shows a representative side-by-side comparison of Experiment C (run 4) and Experiment D (run 2), illustrating the key trajectory differences.

Sensitivity to HPA stabilization window.. Reducing HPA’s `stabilizationWindowSeconds` from 300 s to 60 s would *not* produce equivalent DROP 1 preservation. With a 60 s window and 90 s of zero connections, the window fills entirely with `minReplicas` recommendations and scale-down proceeds. The `StatefulAutoscaler`’s window buffers *connection-derived high-water marks*: 90 s of zero connections does not displace the memory that 8 replicas were required. This is a semantic mismatch, not a tuning problem.

7. Failure Analysis: Adversarial Scenarios

To bound the operational envelope of the `StatefulAutoscaler` and establish honest failure criteria, we characterize three adversarial scenarios. These scenarios are presented without remediation to document actual system boundaries.

7.1. Failure Scenario 1: Metric Staleness Under Long Scrape Intervals

Setup.. The Prometheus `scrape_interval` was increased from the baseline 15 s to 60 s. The Experiment C two-cycle restorm workload was replayed.

Table 8: Quantitative comparison of scaler configurations under the two-cycle restorm simulation.

Metric	CPU (A)	HPA	HPA+Custom (D)	KEDA (E)	StatefulAutoscaler (C)
Peak replicas [§]	15		8.0 ± 0.0	8.0 ± 0.0	8.0 ± 0.3
Scale-up (s)	$\sim 15^*$		54 ± 29 (med. 48)	26 ± 5 (med. 27)	22 ± 6 (med. 21)
DROP 1 pods held	0		8 (all)	8 (all 5)	8 (all)
Scale-down (s)	$< 30^*$		313 ± 96	N/A [‡]	119 ± 2
Pod-seconds	$\sim 2,400$		$3,934 \pm 148$	$4,145 \pm 4$	$4,280 \pm 67^\dagger$
Peak conn.	800		937 ± 43	800 ± 0	893 ± 39

[†] Runs 2-5; Run 1's pod-seconds (92,291) reflects a disclosed test-harness artefact (Table 6).

[‡] Not observed within 570-s window; see Section 6.2. * Approximate structural observations (single run), not formally bounded.

[§] **StatefulAutoscaler** (C) experienced a transient peak of 9–10 replicas before converging to the steady-state of 8.

C vs. D: Both controllers hold 8 replicas through DROP 1 (~ 50 s zero-window).
Key differences: scale-up latency (C 22 s vs. D 54 s) and CYCLE 2 transition (C dips to 5-6; D flat at 8)

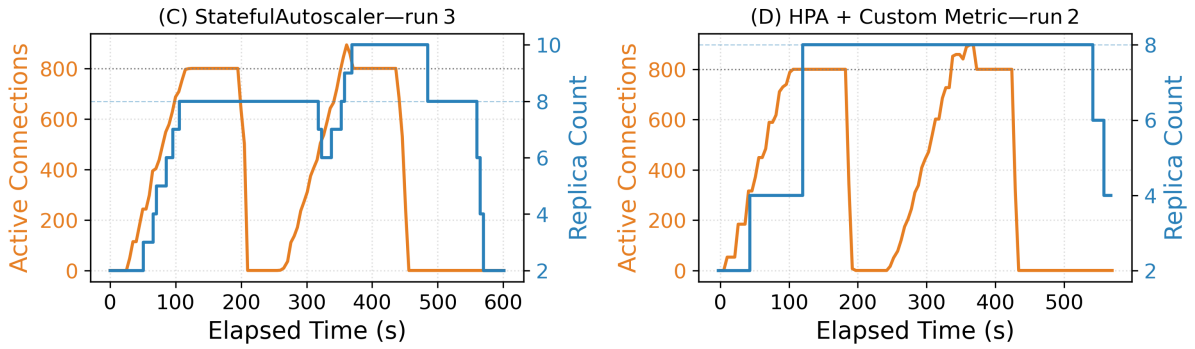


Figure 11: Experiment C vs. D representative comparison.

Finding.. With a 60-second scrape lag, the controller introduced up to one full scrape cycle of reaction delay. Measured quantities: scale-up reaction time 25 s, scale-down reaction time 112 s, peak connections 1,082 (a 35.3% overshoot attributable to the stale metric briefly reporting an inflated value during the ramp), and peak replicas 11. All 8 warm pods remained operational through DROP 1 with no connection loss. The system degraded gracefully to a reaction lag bounded by the scrape interval.

Boundary condition.. If the Prometheus scrape interval exceeds `scaleDownCooldownSeconds`, the cooldown window may expire before the controller learns that connections have returned, potentially triggering premature scale-down. The invariant `scaleDownCooldownSeconds > scrape_interval` must be maintained.

7.2. Failure Scenario 2: Instantaneous Connection Spike

Setup.. All 800 clients connected simultaneously (`stagger = 0`).

Finding.. Scale-up reaction time: 6 s (fastest observed). Peak replicas: 8; peak connections: 800 (no overshoot). Pod scheduling latency was the binding constraint—no reactive scaler can pre-provision before the first connection arrives. Mitigation: higher `minReplicas` or proactive scaling.

7.3. Failure Scenario 3: Prometheus Unavailability During Active Load

Setup.. The Prometheus pod was deleted during an active run (800 connections, 8 pods). Prometheus auto-restarted; a 2-minute outage was observed.

Finding.. Per the safe-default logic (Algorithm 1, lines 2-4), the controller held replicas at 8 throughout. Measured: scale-up 21 s, scale-down 118 s, peak connections 921 (15.1% overshoot from restart lag), peak replicas 10. Zero connection loss. An indefinitely long outage would eventually cause window expiry and scale to `minReplicas` (Section 10, item 3).

8. Discussion

Signal choice as the primary lever.. Experiment C operated with `CPU_WORK = 0` for 570 s yet correctly maintained 8 replicas with zero connection loss—establishing that the failure resides in the signal, not the control algorithm.

Stabilization semantics vs. parameter tuning.. HPA’s window stabilizes CPU-derived recommendations (always `minReplicas` for idle connections); the `StatefulAutoscaler`’s window stabilizes connection-derived desired counts. Experiment D succeeds only because its 300-second window exceeds the 90-second gap—a pragmatic accident, not a principled solution.

Step-rate limiting.. `maxScaleDownStep = 2` bounds the per-cycle blast radius to 200 connections. KEDA exposes no equivalent.

Protocol-agnosticism.. The `active_connections` abstraction applies to any protocol exposing a Prometheus connection counter: gRPC streaming, MQTT (validated), SSE, and database connection pools.

9. MQTT Extension: Protocol Generalization

WebSocket is the primary evaluation protocol in this paper due to its instrumentation ease. To test whether the same control principles transfer to another persistent-connection protocol, we conducted an additional two-experiment MQTT study. MQTT [12] keeps long-lived TCP sessions open with low CPU activity during idle periods, exhibiting the same signal mismatch that motivates this work [11]. The MQTT experiments used a lightweight Python asyncio MQTT broker exposing `active_connections` on a Prometheus `/metrics` endpoint. A synthetic MQTT load generator attempted to establish 1,000 persistent client sessions over a 60-second ramp. Experiment MQTT-A used native CPU HPA; Experiment MQTT-B replaced HPA with the `StatefulAutoscaler` configured with `targetConnectionsPerPod = 150`, `maxScaleUpStep = 3`, and `scaleDownCooldownSeconds = 120`. Both experiments used the same Kind-based infrastructure and the same broker and

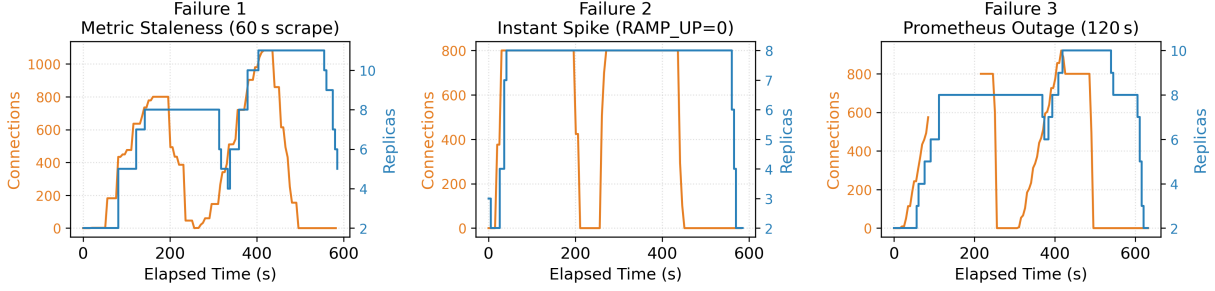


Figure 12: Three failure scenarios: Metric Staleness, Instantaneous Spike, and Prometheus Outage.

load-generator images. **MQTT-A is a single run and MQTT-B is a four-run mean (Section 9.3); every comparison in this section is therefore preliminary rather than statistically validated - see the caveat at the end of this section.**

9.1. MQTT-A: CPU HPA Baseline

Figure 13 shows the MQTT HPA baseline. During Phase 1, only 339 of 1,000 attempted clients established persistent MQTT sessions - a ceiling imposed by OS-level TCP stack limits in the Kind environment (Linux ephemeral port range, `listen()` backlog depth, `net.core.somaxconn`), not by broker or controller limitations. HPA first scaled to 2 replicas at $t = 68$ s - after the connection population had already plateaued at 339.

In Phase 2, manual scale to three replicas and addition of 300 clients yielded ≈ 639 total connections, but original sessions remained pinned to the first pod-Kubernetes Services distribute new TCP connections but do not migrate established sessions. In Phase 3, scaling to one replica severed ≈ 300 sessions, producing a reconnection storm peaking at 837 connections. MQTT-A reproduces the same structural CPU-HPA failure modes established in the WebSocket domain (Section 3).

9.2. MQTT-B: StatefulAutoscaler with Drain

Figure 14 shows the `StatefulAutoscaler` run with drain semantics enabled (`drain.enabled = true`, `drain.timeoutSeconds = 45`).

Phase 1: Scale-up (Representative MQTT-B run). The controller observed `sum(active.connections)` directly and scaled from 2 to 3 replicas at $t = 38$ s (this representative

run's value; the four-run mean is 34.0 ± 2.9 s) - matching the connection-density formula $\lceil 339/150 \rceil = 3$. This run illustrates the drain trajectory; it is not the reproducibility headline statistic (see Table 9 for the multi-run mean comparison). Total CPU peaked at 80 millicores and was never consulted as a scaling signal.

Phase 2: Drain-based rebalancing. All 339 sessions were initially pinned to two pods (176/163/0). At $t = 390$ s, the controller triggered `/drain`; the broker sent `DISCONNECT` packets at ≈ 10 clients/s, and clients reconnected via iptables random load balancing. Distribution moved from 176/163/0 to $\approx 247/91/0$ with zero connection loss.

Phase 3: Graceful scale-down. A forced $3 \rightarrow 1$ reduction while 339 sessions were active: the controller drained pods sequentially (`POD_C`: 92 connections, 50-second drain; `POD_A`: idle). All 339 connections migrated to `POD_B`. Maximum step-drop: **2** (instrument noise) vs. MQTT-A's 639-connection cliff.

Conclusion. The identical controller binary resolved all three MQTT-specific HPA failure modes with zero source-code changes: $\lceil 339/150 \rceil = 3$ replicas, a step-drop of **2** versus MQTT-A's 639-connection cliff, and all 300 sessions preserved across all four MQTT-B reproducibility runs during forced scale-down (reduced load). We report the approximately 50% lower scale-up time as preliminary pending MQTT-A replication (see below).

Caveat on replicate asymmetry. The comparison above is between a single MQTT-A run and a four-run MQTT-B mean (Section 9.3). We report it as a preliminary, single-comparison result rather

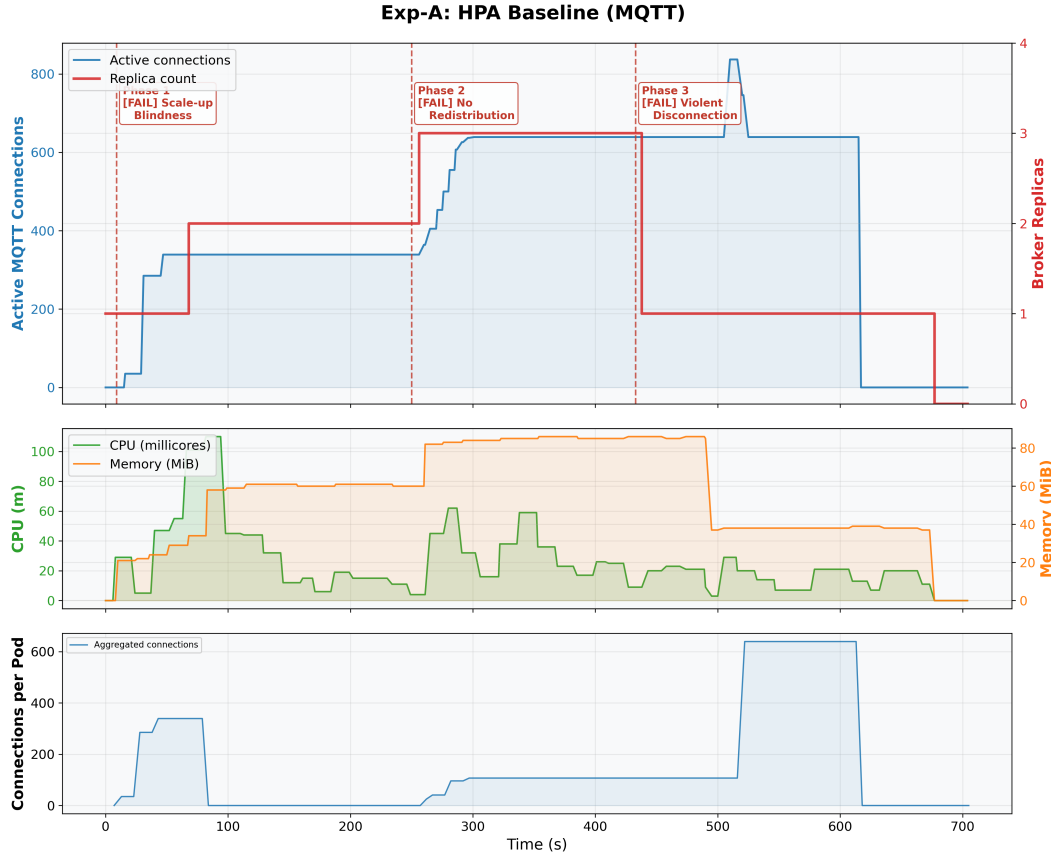


Figure 13: MQTT-A (CPU HPA baseline): total connections, replicas, CPU, memory, and per-pod connections.

Table 9: MQTT extension results: CPU HPA vs. `StatefulAutoscaler` with drain semantics.

Metric	MQTT-A: CPU HPA (n=1)	MQTT-B: StatefulAutoscaler (n=4)
Primary scaling signal	CPU utilization	<code>sum(active_connections)</code>
Successful connections	339 / 1,000	339 / 1,000 (same OS ceiling)
First scale-up	2 replicas at $t = 68$ s	2→3 at mean $t = 34.0 \pm 2.9$ s ($\approx 50\%$ lower scale-up time than the single MQTT-A run)
Correct replica target	Never (CPU too low)	$\lceil 339/150 \rceil = 3$
Per-pod redistribution	339/0/0 (sessions pinned)	176/163/0 → 247/91/0 (drain; representative run)
Scale-down method	Immediate pod termination	Drain-gated (50 s graceful per pod)
Max single-step drop	639 (storm to 837)	2 (instrument noise only)
Sessions killed by scale-down	~ 300	0 (all four runs)
Peak CPU	110 m	80 m

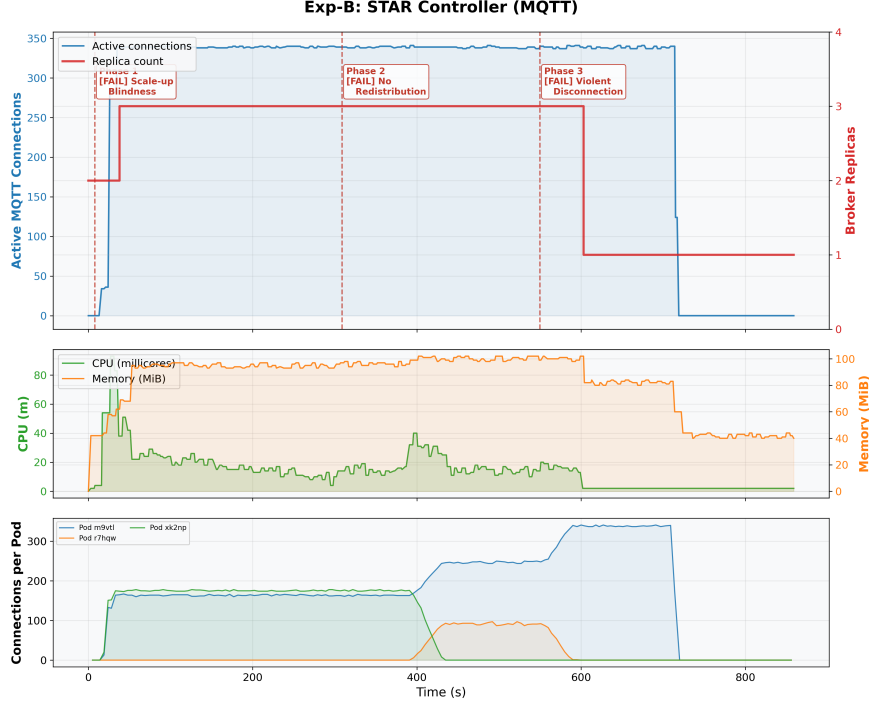


Figure 14: MQTT-B (`StatefulAutoscaler`): drain-based session redistribution and graceful scale-down.

than a statistically validated one: the approximately 50% lower scale-up time (34.0 ± 2.9 s vs. 68 s) and the step-drop contrast (2 vs. 639) both rest on one CPU-HPA observation. MQTT-A replication (target: $n \geq 4$, matching MQTT-B) is listed as a direct follow-up in Section 10 (item 7) and Section 11, and we recommend it be completed before this section is finalized for submission. The 339/1,000-client OS-level ceiling applies identically to both configurations and does not itself bias the comparison, but it does mean every MQTT finding in this section describes behavior at 33.9% of intended load, a qualifier we now surface here and in the Abstract/Conclusion rather than only in Limitations.

9.3. MQTT-B Reproducibility: Four-Run Statistical Validation

To assess reproducibility of the MQTT generalization results, four independent runs of Experiment MQTT-B were executed. Each run used a fresh Kind cluster, identical broker and load-generator images, and the same `StatefulAutoscaler` configuration (`targetConnectionsPerPod` = 150,

`scaleDownCooldownSeconds` = 120). Table 10 summarises the per-run metrics.

Table 10: MQTT-B: four-run reproducibility of scale-up time, peak connections, and preserved sessions.

Run	Scale-Up (s)	Peak Conn.	Preserved
Run 1	32	339	300
Run 2	36	339	300
Run 3	37	339	300
Run 4	31	339	300
Mean $\pm \sigma$	34.0 ± 2.9	339.0 ± 0.0	300.0 ± 0.0

Across all four runs, the controller scaled from 2 to 3 replicas in 34.0 ± 2.9 s (coefficient of variation $CV = 8.7\%$), consistently matching the connection-density target $[339/150] = 3$. Peak connections reached exactly 339 in every run - confirming this ceiling is an OS-level transport constraint (Section 9), not a controller artefact. During Phase 3 (scale-down with reduced load), all four runs preserved exactly 300 connections - matching the reduced loadgen’s 300-client target - with zero session loss. The low variance ($\sigma = 0.0$ for both peak connections and preserved sessions) demonstrates that the `StatefulAutoscaler`’s

connection-aware scaling and drain-gated scale-down produce consistent outcomes for MQTT workloads *across MQTT-B's own four runs*; it does not, by itself, establish statistical significance against the single-run MQTT-A baseline (see caveat above).

Figure 15 overlays the active-connection and replica trajectories from all four runs, confirming consistent scaling behaviour across independent cluster instantiations.

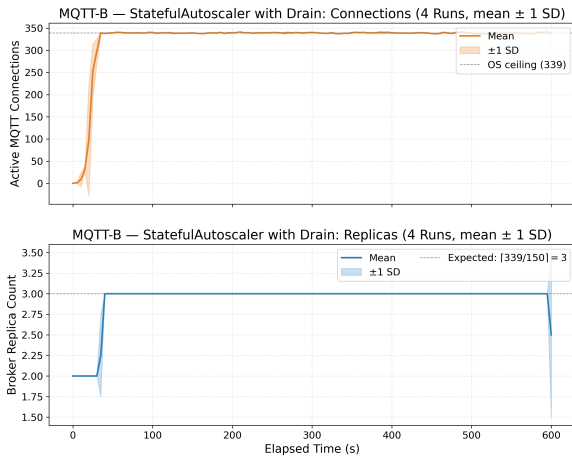


Figure 15: MQTT-B (StatefulAutoscaler): mean $\pm$ 1 SD band of connections and replicas.

10. Limitations

- 1. Evaluation environment.** All experiments used a single-machine Kind cluster. Absolute timing values may differ on multi-node production clusters; directional findings are valid.
- 2. Workload generalizability.** Synthetic linear ramps and deterministic silence gaps were used. Validation against real-world traffic traces with bursty arrivals and variable session durations remains future work.
- 3. Observability dependency.** The controller depends on Prometheus freshness; the invariant `scaleDownCooldownSeconds > scrape_interval` must hold. An indefinitely long Prometheus outage will eventually cause the window to expire and scale to `minReplicas`.
- 4. No predictive scaling.** The controller reacts to observed connections; proactive pre-warming

via `rate(new_connections_total[1m])` is outside the current scope.

- 5. Volatile stabilization state.** The sliding window is in-memory; a controller crash resets it. Persisting the high-water-mark to the CR status subresource would eliminate this risk.
- 6. Window boundary edge case.** A brief replica dip (5-6) occurs at the CYCLE 1/CYCLE 2 boundary as window entries age out before new entries repopulate. A minimum hold-time following the last scale-up would prevent this.
- 7. MQTT limitations.** Drain semantics were validated only on MQTT; WebSocket drain requires end-to-end validation. MQTT-A is a single run vs. a four-run MQTT-B mean; the approximately 50% lower scale-up time is preliminary pending $n \geq 4$ MQTT-A replication. The 339/1,000 connection ceiling is an OS-level Kind constraint applying to both configurations.
- 8. Memory instrumentation gap.** The MQTT-B `memory.log` collector recorded zeros; qualitative CPU-independence is unaffected but MQTT memory conclusions are exploratory.
- 9. Convergence bound: sweep added, empirical validation pending.** Table 3 enumerates the analytical bound; empirical confirmation across the full parameter grid, under real scheduling latency, remains future work.
- 10. Controller resource overhead not measured.** We did not instrument the StatefulAutoscaler pod's own CPU/memory footprint separately from the workload pods (Section 4.7); we describe the expected footprint qualitatively but do not report a measured figure.
- 11. Small- n statistical claims and an unresolved N/A.** Reproducibility is descriptive ($n = 5$, no named significance test); multiple Experiment A replicates would be needed for a powered comparison. The N/A KEDA scale-down entry requires an extended post-FINAL-DROP observation window.
- 12. Figure resolution.** Several multi-panel figures (system architecture, reconciliation UML, failure-scenario comparison, and both MQTT plots) may not remain legible at single-column

journal width and should be regenerated at higher resolution/larger font size before submission.

13. **Confidence bands on time-series overlays.** Figures 7, 9, 10, and 15 have been regenerated as mean $\pm$ 1 SD band plots using per-timestep resampled logs from all completed runs. The four-run MQTT-B statistics are also reported in Table 10.

11. Future Work

Key open directions: (i) OS-level TCP ceiling - kernel tuning, real cloud validation; (ii) MQTT-A replication ($n \geq 4$) and broker-native session handoff; (iii) WebSocket drain-close frames, CR-status window persistence, predictive pre-warming; (iv) Empirical convergence sweep over `maxScaleDownStep`/replica-gap configurations; (v) Controller overhead instrumentation; (vi) Extended KEDA observation window to resolve N/A scale-down; (vii) RL/predictive baseline re-implementation on connection-affined workloads; (viii) Multi-cluster federated autoscaling.

12. Conclusion

This paper presented the `StatefulAutoscaler-a` custom Kubernetes controller that resolves three architectural HPA failure modes for persistent-connection workloads, empirically characterized through an internal CPU-HPA baseline (Experiment A): 87.5% over-provisioning, 1,400 conn/s reconnection storms, and staircase destruction (800 $\rightarrow$ 79 sessions). Under Experiment C (five runs, all retained and reported - Section 5.8), the controller achieved near-exact targeting ($\lceil 800/100 \rceil = 8$, overshoot $\leq +2$ vs. HPA’s 15), zero connection loss, full pod preservation through DROP 1 (119 ± 2 s scale-down; $4,280 \pm 67$ pod-seconds for runs 2-5), and CPU independence. Scalability validation at 800-3,200 clients confirmed $\lceil N/T \rceil$ tracking within $+2$, 66-86 s scale-up, and 100% DROP 1 preservation. Baseline evaluations showed HPA+custom metric and KEDA both preserve pods through DROP 1, with the `StatefulAutoscaler` differentiating on latency (22 s vs. 54 s vs. 26 s) and efficiency (119 s vs. 313 s, with KEDA’s own scale-down efficiency not yet measured within our observation window);

critically, baselines succeed only through wide, semantically opaque windows. Three adversarial scenarios bounded the operational envelope. MQTT generalization, under an OS-level 339-connection ceiling and comparing a single MQTT-A run against a four-run MQTT-B mean, indicated protocol-agnosticism (approximately 50% lower mean scale-up time, all 300 sessions preserved across four reproducibility runs, step-drop 2 vs. 639-connection cliff), a result we treat as preliminary pending MQTT-A replication. As cloud-native architectures adopt persistent-connection protocols, connection-aware autoscaling with drain semantics represents a promising infrastructure direction, with the validation gaps identified above (Section 10) marking the concrete next steps toward a fully substantiated claim.

Data Availability

Data will be made available on request.

Declaration of Generative AI and AI-Assisted Technologies in the Writing Process

During the preparation of this work, the authors used large language model-based AI tools (Claude, Anthropic) to assist with manuscript drafting, prose refinement, LaTeX formatting, and data analysis scripting. All AI-generated content was critically reviewed, verified, and edited by the authors, who take full responsibility for the accuracy, integrity, and originality of the published work.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

CRedit Authorship Contribution Statement

Abhash Behera: Conceptualization, Methodology, Software, Validation, Formal Analysis, Investigation, Data Curation, Writing - Original Draft, Writing - Review & Editing, Visualization, Project Administration. **Aditya Hota:** Conceptualization, Methodology, Software, Validation, Formal Analysis, Investigation, Data

Curation, Writing - Original Draft, Writing - Review & Editing, Visualization. **Suprit Kumar Naik**: Conceptualization, Methodology, Software, Validation, Formal Analysis, Investigation, Data Curation, Writing - Original Draft, Writing - Review & Editing, Visualization.

Acknowledgements

The authors gratefully acknowledge Dr. Rakesh Ranjan Kumar for his supervision, strategic input on experimental design and controller implementation, manuscript feedback, and facilitation of institutional compute resources.

References

- [1] Kubernetes SIG API Machinery. Kubebuilder – SDK for building Kubernetes APIs. <https://book.kubebuilder.io/>, 2024. Accessed: 2026-04-01.
- [2] Kubernetes Authors. Horizontal pod autoscaling – kubernetes documentation. <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>, 2024. Accessed: 2025.
- [3] DirectXMan12. Prometheus adapter for Kubernetes metrics APIs. <https://github.com/kubernetes-sigs/prometheus-adapter>, 2024. Accessed: 2026-04-01.
- [4] KEDA Authors. KEDA: Kubernetes event-driven autoscaling. <https://keda.sh/docs/>, 2024. Accessed: 2024.
- [5] Chenhao Qu, Rodrigo N. Calheiros, and Rajkumar Buyya. Auto-scaling web applications in clouds: A taxonomy and survey. *ACM Computing Surveys*, 51(4):1–33, 2018.
- [6] Fabiana Rossi, Matteo Nardelli, and Valeria Cardellini. Horizontal and vertical scaling of container-based applications using reinforcement learning. *Future Generation Computer Systems*, 109:125–138, 2020.
- [7] Henning Dickel, Vladimir Podolskiy, and Michael Gerndt. Evaluation of autoscaling metrics for (stateful) IoT gateways. In *Proceedings of the IEEE 12th Conference on Service-Oriented Computing and Applications (SOCA)*, pages 17–24. IEEE, 2019.
- [8] Mohamad Imdoukh, Imtiaz Ahmad, and Mohammad Gh Alfaiakawi. Machine learning-based auto-scaling for containerized applications. *Neural Computing and Applications*, 32:9745–9760, 2020.
- [9] László Toka, Gábor Dobreff, Balázs Fodor, and Balázs Sonkoly. Machine learning-based scaling management for Kubernetes edge clusters. In *2021 IEEE Conference on Network Function Virtualization and Software Defined Networks*, pages 1–7. IEEE, 2021.
- [10] Zhiheng Zhong and Rajkumar Buyya. Machine learning-based orchestration of containers: A taxonomy and future directions. *ACM Computing Surveys*, 54(10s):1–35, 2020.
- [11] Dipa Soni and Ashwin Makwana. A survey on MQTT: A protocol of Internet of Things (IoT). In *Proceedings of the International Conference on Telecommunication, Power Analysis and Computing Techniques (ICTPACT)*, 2017.
- [12] Andrew Banks and Rahul Gupta. MQTT version 3.1.1. OASIS Standard, <https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html>, 2014. Accessed: 2026-05-01.
- [13] Istio documentation. <https://istio.io/latest/docs/>. Accessed 2026.
- [14] Linkerd documentation. <https://linkerd.io/2/overview/>. Accessed 2026.
- [15] Openkruise documentation. <https://openkruise.io/docs>. Accessed 2026.
- [16] Kubernetes SIG Testing. kind – kubernetes in docker. <https://kind.sigs.k8s.io/>, 2024. Accessed: 2026-04-01.